Multi-Agent AI Workflow Engine

Simulating an Autonomous Software Development Team

Marco Perozo • Gabriel Marte • Lewdy Aquino • Alvin Almonte

Department of Computer Science · Southwest Minnesota State University · Capstone Project, Fall 2026

Introduction & Problem Statement

Large Language Models (LLMs) can now generate source code from natural-language descriptions, and tools such as GitHub Copilot and ChatGPT are used daily by developers (Chen et al., 2021). Almost all of them are built around one model receiving one prompt: a single pass in which the model must understand the request, design the solution, write the code and verify it all at once.

Three limitations follow:

- Single-model limits. One pass over a large, multi-step task produces incomplete or incorrect results on anything non-trivial.
- Lack of teamwork. Real software needs a product manager to clarify, a developer to build and QA to verify. A single prompt collapses these perspectives and loses the checks and balances that make teams effective.
- Lack of iteration. Standard prompts do not reliably self-correct against predefined requirements (Madaan et al., 2023); detecting and fixing errors falls to the human.

RESEARCH QUESTION

Does a structured, role-based multi-agent workflow (PM → Developer → Reviewer with an iteration loop) produce more correct and more requirement-compliant software than a single LLM given the same task — and at what cost in tokens and time?

Goal & Objectives

Goal. Design, implement and evaluate an engine that simulates an autonomous software development team and turns a simple prompt into working, reviewed code through a controlled iteration loop.

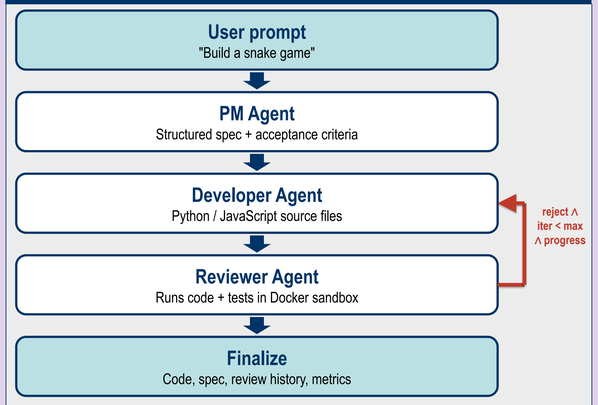
- Build the orchestration pipeline as a LangGraph state graph.
- Implement PM, Developer and Reviewer agents, each producing a well-defined artifact (spec, code, review report).
- Engineer loop prevention so the review/fix cycle always terminates.
- Implement memory management (rolling summarization and pruning).
- Persist every run, message, spec, code version, review and metric.
- Evaluate against a single-model baseline on ≥ 20 hidden-test tasks.

Related Work

Agentic frameworks show that decomposing a problem across cooperating LLMs improves quality on complex tasks: AutoGen provides a general multi-agent conversation layer (Wu et al., 2024); MetaGPT encodes standard operating procedures for software roles (Hong et al., 2024); ChatDev organises agents into a virtual software company (Qian et al., 2024). Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2023) show that iterative self-feedback raises output quality, and SWE-bench (Jimenez et al., 2024) supplies a rigorous hidden-test evaluation protocol.

Gap addressed. These systems report capability, but rarely report guaranteed termination or cost per successful task. MAWE makes the control flow explicit and testable, and measures cost-per-success directly.

System Architecture



Six layers. Orchestration (LangGraph state graph) · Agent (role prompts and tools) · Safety (iteration caps, exit conditions, fallback routing) · Memory (summarization and pruning) · Persistence (SQLAlchemy / SQLite → PostgreSQL) · Evaluation (benchmarks against a single-model baseline).

Methodology

Development process. Four two-week Agile sprints; every change goes through a GitHub pull request reviewed by a teammate; GitHub Actions runs the unit-test suite on each push; 15-minute weekly stand-ups on a Kanban board.

Workflow as a state graph. Nodes are agent steps (plan, develop, review, summarize, finalize); edges are routing rules evaluated on the current state, which makes the control flow explicit, testable and easy to visualise.

START → PM.plan → Dev.implement → Reviewer.review → [approve → finalize] | [reject ∧ iter < max ∧ progress → Dev.implement] | [reject ∧ (iter = max ∨ no progress) → finalize_with_warnings] → END

Testing strategy. Unit tests for routing, loop-prevention rules, summarization and database access; integration tests over the full pipeline with a mocked LLM for determinism; end-to-end runs with real LLM calls before each milestone.

Loop Prevention & Memory Management

The two failure modes that break autonomous agent pipelines are non-termination and context growth. Both are engineered against explicitly.

Four termination mechanisms

- Hard iteration cap — configurable, default 5.
- Severity threshold — only critical and major issues cause rejection; minor issues are reported but do not block approval.
- No-progress detection — a SHA-256 hash of the Developer output exits the loop when the same code or the same issue list repeats.
- Fallback routing — on exit without approval the engine finalizes the best version with a warnings report rather than failing.

Memory strategy

Only the latest spec, latest code and latest review stay in full context; older iterations are compressed into a compact progress log stored in memory_summaries, while every raw message is still persisted in the database so nothing is lost for later analysis.

Database Design

The database is the system's memory of record: every run is fully traceable from the original prompt through each code version to the final metrics. The schema is in third normal form, written with SQLAlchemy so SQLite (development) migrates to PostgreSQL unchanged.

Table	Purpose
users / projects	Owner and the original idea prompt
agents	Role, model, system prompt, version
runs	Mode, status, iterations, tokens, cost
messages	Every agent-to-agent exchange
specifications	Versioned PM spec + acceptance criteria
code_artifacts	Each file per iteration + SHA-256 hash
reviews	Verdict, severity, tests passed/failed
memory_summaries	Compressed context, tokens before/after
benchmark_tasks	Prompt, difficulty, hidden tests
evaluation_results	Metric name and value per run

USER ——< PROJECT ——< RUN ——< { MESSAGE >—— AGENT, SPECIFICATION, CODE_ARTIFACT, REVIEW, MEMORY_SUMMARY, EVALUATION_RESULT } · BENCHMARK_TASK ——< RUN

Implementation Status

Stack. Python 3.11+ · LangGraph · OpenAI / Anthropic / Google API SDKs · Docker sandbox with no network and CPU, memory and time limits · pytest and Jest · SQLAlchemy over SQLite → PostgreSQL · FastAPI · Git and GitHub Actions. No GPU is required: all models run in the cloud via API.

Phase	Weeks	Milestone	Status
1. Setup & prototyping	1–2	M1 pipeline running	M1 met
2. Role implementation	3–5	M2 full PM/Dev/QA loop	Planned
3. Memory & state	6–8	M3 engine v1.0	Planned
4. Testing & evaluation	9–12	M4 benchmark + report	Planned

Experimental Evaluation

Design. The same benchmark tasks are solved under four conditions using the same underlying model and temperature, so that any difference is attributable to the workflow rather than the model.

- A — Single model (baseline): one call, one answer.
- B — Single model with self-refinement: the same model writes, then reviews and fixes its own code under the same iteration cap.
- C — MAWE (proposed): PM → Dev → Reviewer with loop prevention and memory management.
- C' — Ablation: MAWE with summarization disabled.

Benchmark. ≥ 20 Python and JavaScript tasks with hidden tests the agents never see — 7 easy, 8 medium, 5 hard. Three repetitions per task per condition (≈ 240 runs) to account for LLM non-determinism; means reported with 95% confidence intervals and tested with a paired t-test or Wilcoxon signed-rank test.

Metrics. Hidden-test pass rate, task success rate, specification coverage, iterations used, total tokens, estimated cost, latency, termination reason and context reduction from summarization.

Preliminary Results

Milestone M1 (Week 2). The engine runs end to end against a scripted mock LLM, so the orchestration, loop-prevention and persistence layers can be measured with no API cost and with repeatable results. These numbers characterise the ENGINE, not model quality; code-quality benchmarks against real LLMs run in Phase 4.

Scenario	Iter.	Termination	Tokens	Calls
Reject, reject, approve	2	approved	1,002	6
Developer returns identical code	2	no_progress	863	5
Reviewer never approves	5	iteration_cap	2,099	12

3 / 3 runs terminated — no hang, no manual stop. All three exit paths fire: severity threshold, code-hash no-progress detection and the hard iteration cap. 20 messages, 3 specifications, 9 code artifacts and 8 reviews persisted to SQLite and queryable per run.

Phase 4 template	Pass rate	Task success	Cost / success
A — Single model	—	—	—
B — Self-refinement	—	—	—
C — MAWE	—	—	—
C' — No summarization	—	—	—

Discussion & Conclusions

MAWE is expected to improve correctness and requirement coverage, most visibly on medium and hard tasks where a single prompt tends to miss requirements or ship untested bugs. The price is higher token usage and latency, since each iteration adds at least two model calls. The question is therefore not "is it better?" but "is the improvement worth the cost?" — which cost per successful task answers directly.

Condition B isolates iteration from role separation. If C still wins, that is evidence that an independent reviewer is worth more than a model checking its own work — the same reason real teams separate development from QA.

Limitations. The Reviewer is itself an LLM and can approve incorrect code, which is why scoring uses hidden tests instead of its verdict; tasks are small to medium, so conclusions may not transfer to large multi-module systems; results depend on the model family; and the budget limits repetitions to three per task.

References

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., … Zaremba, W. (2021). Evaluating large language models trained on code. arXiv:2107.03374.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., … Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. International Conference on Learning Representations (ICLR 2024).

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? International Conference on Learning Representations (ICLR 2024).

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., … Clark, P. (2023). Self-Refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems, 36.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., … Sun, M. (2024). ChatDev: Communicative agents for software development. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 15174–15186.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., … Wang, C. (2024). AutoGen: Enabling next-generation LLM applications via multi-agent conversation. Conference on Language Modeling (COLM 2024).

Acknowledgements

We thank Dr. Hezy for supervising this capstone project and for feedback on the proposal and evaluation design, and the maintainers of the LangGraph and CrewAI open-source projects whose tooling this work builds on.

POSTER TEMPLATE BY
www.PosterPresentations.com